

ProDiligix Logistics Management System

System Architecture Document

Project Overview

ProDiligix LMS is a web based logistics management system designed to manage customers, shipments, tracking, reports and business analytics.

The application follows a modern three-tier architecture consisting of Frontend Layer, Backend Layer and Database Layer.

The system provides secure authentication, role based access control and real-time logistics operations management.

Technology Stack:

Frontend:

- React.js
- Vite
- Tailwind CSS
- Axios
- Recharts

Backend:

- Node.js
- Express.js
- JWT Authentication
- Bcrypt Password Encryption

Database:

- PostgreSQL

User Roles:

1. ADMIN

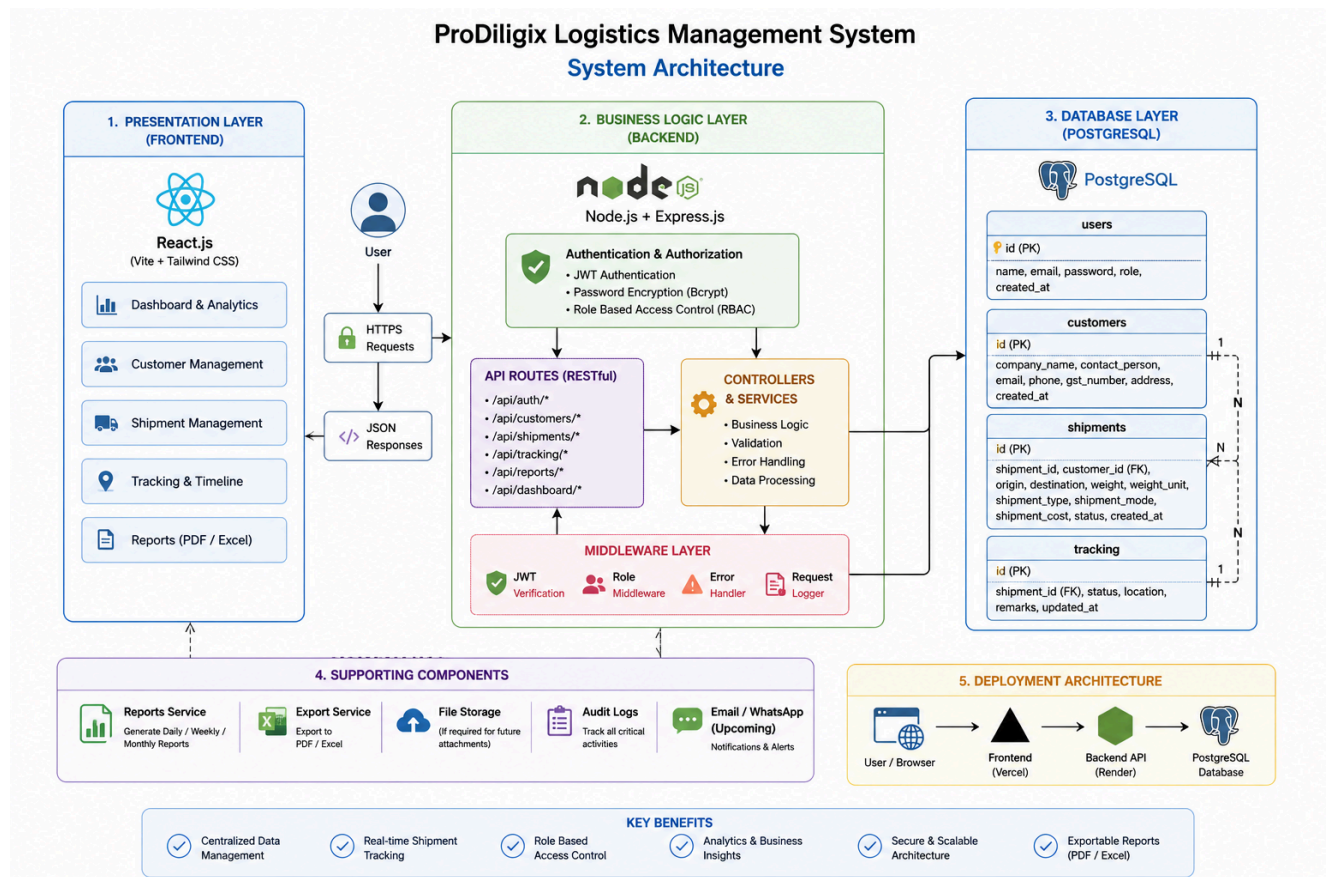
- Complete system access
- Manage customers
- Manage shipments
- View dashboard
- Generate reports

2. OPERATIONS

- Customer management
- Shipment creation
- Shipment tracking updates

3. MANAGEMENT

- Dashboard analytics
- Reports access
- Tracking visibility



Architecture Explanation

1. Presentation Layer (Frontend)

The frontend layer is developed using React.js.

Responsibilities:

- User interface rendering
- Dashboard analytics visualization

- Customer management screens
- Shipment management screens
- Tracking timeline
- Report export functionality

Frontend communicates with backend APIs using Axios HTTP requests.

2. Business Logic Layer (Backend)

The backend layer is developed using Node.js and Express.js REST APIs.

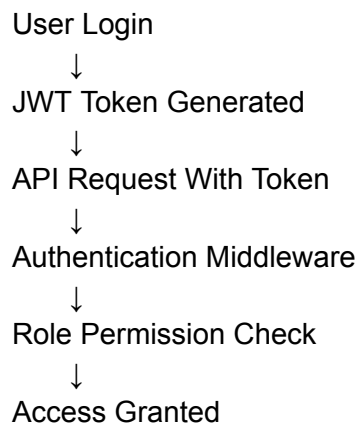
Responsibilities:

- API request handling
- Authentication
- Authorization
- Business logic processing
- Database communication
- Error handling

Security Implementation:

JWT Authentication is used for secure user sessions.

Flow:



3. Database Layer

PostgreSQL database is used for storing application data.

Main Tables:

Users:

Stores login credentials and roles.

Customers:

Stores customer/company information.

Shipments:

Stores shipment details including origin, destination, cost and status.

Tracking:

Stores shipment status updates and history.

Database Relationship:

One Customer



Multiple Shipments

One Shipment



Multiple Tracking Updates

Deployment Architecture:

User Browser



React Frontend



Express Backend API



PostgreSQL Database

Conclusion:

The architecture ensures scalability, security and maintainability by separating frontend, backend and database responsibilities.

Role based access control provides secure enterprise level permissions for different users.